A simulation of the test is not an implementation of the wiring

Abstract

A finite process function is a wiring in which every replacement of the local maps has exactly one history. A public theorem says that a canonical source-sink permutation stays unitary, under every local unitary and every matched finite ancilla, if and only if the wiring is a process function. A machine can search an explicit table and report the count of histories. That search can be implemented, in the sense that a physical device's causal transitions mirror the search's formal steps. The search does not thereby implement the wiring. The permutation is a formal relabeling, defined for wirings that fail the fixed-point test as well as for those that pass. A simulator that prints the right counterfactuals about the table supports those counterfactuals. Support of that kind is not the isomorphism between formal transitions and causal transitions that an implementation condition requires. Nothing follows about consciousness, mentality, or the moral status of any machine. The mathematics is an alternative adaptation of results already public.

Keywords: implementation; simulation; computation; process functions; counterfactuals

1 What is not at issue

The finite results are already public. They are not re-proved here, and they are not offered as a theory of mind. A map on finite alphabets is a process function when every tuple of local functions has exactly one fixed point. Call the fixed point a history. A canonical extension $U|o, e\rangle = |w(o) + e, o\rangle$ is a basis permutation for every map w, whether or not w is a process function. The closed loop obtained by inserting local unitaries on matched finite ancillas is unitary for every such insertion if and only if w is a process function. An explicit table can be searched. A known board induces, in one basis, a distribution on which a stated linear functional is negative. Those are the mathematical facts this paper is allowed to use.

The philosophical question is about implementation. Three objects invite the word: the search, the permutation, and the loop. A device that carries out the search is a computer in an ordinary sense. It is tempting to say that the device therefore implements the wiring, or that a unitary description of the loop is already an artificial agent, or that counterfactual agreement between a simulator and the table is realization. The temptation should be refused, and the refusal does not require a view about consciousness.

Chalmers (n.d.) gives a condition for implementation that is useful here and is not, in this paper, a premise for anything about minds. He writes that a physical system implements a given computation when the causal structure of the system mirrors the formal structure of the computation. For a finite-state automaton, the condition he states is a mapping from physical states, inputs, and outputs to formal ones such that each formal transition is mirrored by a reliable causal transition: if the system is in the physical state and receiving the physical input that map to a formal pair, that reliably causes the next physical state and output that map to the formal successors. He adds that a mere mapping of states, of the sort he attributes to Searle, is not enough. The reliable transition rules do the work. He also distinguishes a simulation that shares only a description from a simulation that shares causal topology. On his view, organizationally invariant properties are replicated, not merely simulated, when the topology is shared. Being a hurricane is not organizationally invariant. He argues that mental properties are. This paper does not accept, reject, or extend that last argument. It does not discuss dancing qualia, the Chinese room as an argument about understanding, or the sufficiency of computation for

mentality. Those parts of his article were read far enough to see that they are a different thesis. They are set aside.

What is used is the implementation condition, the warning that a state mapping without reliable transitions is too cheap, and the distinction between matching a description and sharing the causal pattern the description specifies. Applied to the three objects above, the condition says that a search can be implemented without the wiring being implemented, and that a formal unitary can exist without any device whose causal transitions mirror the process-function test.

2 The condition, applied without a theory of mind

Chalmers' finite-state clause is a counterfactual about the device. If the device is in a certain state and receives a certain input, it reliably goes to a certain next state and output. The formal automaton has the same pattern under the mapping. Implementation, on this clause, is not the existence of a function from inputs to outputs. It is the reliable causal holding of the transitions.

A process function is not, by itself, an automaton of that shape. Its defining condition is a fixed point of a simultaneous composition: an output tuple o such that each local function, applied to the input the wiring assigns, returns the corresponding component of o. There is no "old state" and "new state" inside the definition. One can encode the search for that fixed point as an automaton. The encoding has steps. The wiring's definition does not become those steps. Chalmers' condition, applied to the search-automaton, asks whether a device's causal steps mirror the search. Applied to the wiring, it asks a question the wiring has not yet been given the form to answer, unless one specifies which transitions are supposed to be mirrored. Different specifications are different implementation claims.

He notes that every physical system implements some trivial automaton, and that this does not make the account vacuous, because not every system implements every complex computation. The transition constraints are what exclude the trivializing reading. That point matters here in a limited way. A device that sits still implements some one-state machine. It does not, on his condition, implement a search with a large transition table unless its causal steps mirror that table. And a proof that a permutation of basis vectors exists does not state a causal transition of any device. The proof is not an implementation claim of his kind at all.

His further thesis, that a model sharing the causal topology of a mind replicates mental properties, is not a lemma of the implementation condition. It needs the claim that the properties in question are organizational invariants. This paper names no mental property and does not assert that any property of a process function is an organizational invariant in his sense. A reader who wants that assertion will not find an argument for it below. The omission is deliberate. The user of these drafts has asked that no consciousness result be manufactured, and the mathematics does not contain one.

3 The search

Given a finite table for w and a finite list of local functions, a procedure can enumerate output tuples and count solutions of the fixed-point equation. The procedure is a computation in the ordinary sense. Its formal structure is a sequence of steps: read an entry, test an equality, increment a counter, halt. Chalmers' condition says a physical device implements that computation when the device's causal transitions reliably mirror those

steps under some mapping. A laptop that runs the procedure is a candidate. Whether a particular laptop does so is an empirical question about that laptop. Nothing in the public mathematics answers it for any named machine. The mathematics shows only that the procedure exists and halts on explicit tables.

Suppose the laptop does implement the search. What has been implemented is the search. The formal transitions are transitions among the procedure's states: which row is being tested, what the counter holds. They are not transitions among the output tuples of the wiring, except in the trivial sense that the procedure writes a tuple down when it finds one. Writing down a description of a history is not undergoing the history. A device that prints $(0, 0, 0)$ for the three-bit copying case has produced a numeral. Chalmers' condition, applied to a computation whose states are the wiring's output tuples and whose transitions are the local maps, would require reliable causal transitions among those tuples. The printer's transitions are among print states. The mapping that sends "the printer is in the state of having printed $(0, 0, 0)$" to "the history is $(0, 0, 0)$" maps a result to a result. It does not map a causal transition of the wiring to a causal transition of the printer, because the wiring, as defined, has not been assigned those causal transitions.

The same point blocks a complexity reading. The search decides a finite table. It does not show that every family of wirings is decidable in a stated time, and it does not show that the wiring computes a function the search does not compute. Implementation of a decision procedure for a table is not implementation of an oracle for a different problem. That negative claim is all the complexity content this paper needs. There is no speedup theorem in the source mathematics, and none is added.

4 The permutation and the loop

The operator U is defined for every w. It sends a basis vector labeled by an output tuple and an ancilla symbol to a basis vector labeled by the wired input plus the ancilla, and by the output tuple. As a map on a finite set it is a permutation, hence unitary as an operator. Chalmers' condition does not apply to an operator merely by the operator's being unitary. The condition asks for a physical system and a reliable causal pattern. U is not a physical system. A sentence that calls U an implementation has changed the subject from his relation to a mathematical object.

One could try to restore the subject by building a device whose causal transitions permute a set of physical states as U permutes the basis. That would be an implementation of the permutation, if the transitions were reliable and the mapping were in place. It would still not be an implementation of the process-function test. The permutation exists for the copy-and-flip wiring, which has no history, and for the copy-and-copy wiring, which has two. A device that mirrored U for those wirings would mirror a relabeling that does not encode unique solvability. Unique solvability is a fact about fixed points of the composed local maps. The permutation's action does not test those fixed points. A mapping from the device's states to basis labels can be an implementation of the permutation and a failure to implement the test.

The loop theorem is a biconditional about operators. Insert local unitaries, of matching finite dimension, and the loop is unitary for all such insertions exactly when every local-function substitution has one history. The biconditional is a formal mirror of a sort: two mathematical conditions coincide. Chalmers' mirror is between a formalism and a causal structure. A coincidence of two formal conditions is not that mirror. It can be a specification of what a device would have to do if one wanted the device's causal pattern to match the loop. The specification is not the device. A simulator that multiplies the matrices and reports that the product

is unitary is, again, a candidate implementation of the matrix arithmetic, not of the loop as a physical process.

The known-board witness does not change the classification. It is a family of unitaries, a source vector, and a distribution in the computational basis. A program that evaluates the distribution implements, if anything, the evaluation. The distribution's lying outside a convex hull, if an external enumeration is granted, is a mathematical separation. It is not a causal transition in a laboratory, and it is not evidence that a machine is an agent. The basis is named. Another basis need not yield the same separation. A fixed-basis calculation is not a claim that every physical description of the board has the same classical readout.

5 Parts, not a single state

The finite-state clause is the simplest case Chalmers states. He then tightens it. A combinatorial-state automaton has state vectors, and an implementation requires a vectorization of the physical system into components. He calls it a natural requirement that each element correspond to a distinct physical region, with a mapping such that each formal transition rule is mirrored by a reliable causal transition of the whole vector. The added structure is what, on his account, makes a complex computation hard to implement: not a mapping of states alone, but many transition constraints.

That clause is the one a wiring would have to meet if it were read as a combinatorial-state automaton. The parties are the obvious candidates for the vector's components. A local map would have to be a causal dependency among regions, reliable under the stated inputs. The fixed-point condition does not say that. It says which output tuples agree with a simultaneous composition. One can add a story on which each party is a region and each local function is a causal rule. The story is an extra premise. The public theorem does not contain it, and a search that treats the parties as columns of a table does not supply the regions.

The clause also shows why the permutation is a poor stand-in. U acts on one space of basis vectors labeled by whole tuples. It does not exhibit a transition rule for each component under independent inputs. The loop's matrix element is a product of local unitary entries, a formal factorization. A formal factorization is not a vectorization of a physical system into regions. Chalmers' condition would ask for the regions and the reliable transitions. The product formula asks for neither.

He notes that a Turing machine can be redescribed as a combinatorial-state automaton whose vector includes the state of the head and the state of each tape square. That redescription fits the search, whose steps are tape-like. It does not fit the wiring until the wiring is given a head, a tape, and transition rules. The search has those because it was written as a procedure. The wiring does not inherit them from the fact that the search mentions the wiring's table.

6 Counterfactual support

A different relation is easy to confuse with implementation because it also uses counterfactuals. The process-function condition supports counterfactuals inside the model: if the local functions were these, the history would be that one. A simulator that correctly evaluates the table supports the same counterfactuals in the weaker sense that it prints the consequent when given the antecedent. Call that counterfactual support. It is support for a prediction about the model, delivered by a calculation. It is not Chalmers' reliable causal transition in a system whose states are the histories.

The difference shows up when the counterfactual is false of any device and true of the model. The three-bit rule, under the copying functions, has the history $(0, 0, 0)$ as a mathematical fact. A simulator prints that fact. There may be no device whose parts are the three bits and whose interventions are the local functions. The counterfactual "if one set the copying functions, the record would be $(0, 0, 0)$" is then a counterfactual about a device that has not been specified, or is false because there is nothing to set. The simulator's printout remains correct as arithmetic. Counterfactual support of the arithmetic does not fill in the device.

Chalmers' hurricane example, which he uses against the objection that a computational model is "just a simulation," is instructive and limited. He says a computer model of a hurricane is not a hurricane, because hurricanehood requires wind and air, whereas a model that shares the causal topology of an organizationally invariant property replicates that property. This paper does not need to decide whether he is right about cognition. It needs the distinction he draws between sharing a topology and sharing a description. A printout of $(0, 0, 0)$ shares a description with the fixed point. It does not share a causal topology with a three-bit device unless there is such a device and the topology has been shown. The printout is, in his hurricane sense, on the side of the model that is not the thing modeled. That comparison does not insult the printout. It locates it.

There is a benign use of the simulator's counterfactuals. They are what one would compare with records if an intervention test were run. Another adaptation of this mathematics says what that test would have to identify, and says the identifications have not been made. This paper does not repeat that list. It adds that even a perfect simulator, which gets every counterfactual about the table right, implements the simulation. Getting the counterfactuals right is evidence that the simulator matches the model. It is not evidence that the simulator is the system the model describes, and it is not evidence that the model describes any system.

7 Objections

The first objection says that Chalmers' condition is itself too liberal, so using it does not protect the distinction. He agrees that trivial automata are everywhere, and he argues that complex transition tables are not. The search for a three-bit table is a complex table of steps. A rock does not implement it unless, under a mapping, the rock reliably carries out those steps. His answer to the trivializing reading is the answer this objection needs, and it is his answer, from the text, not a new proof that no mapping of a rock exists. If a contrived mapping were found on which a rock's unrelated physical changes satisfied the search's transition condition, that would be a problem for the condition as applied to that mapping. It would not show that the search and the wiring are the same computation. The wiring still lacks the search's step structure until someone adds it.

The second objection says that a unitary operator is already a causal specification, because unitaries are what quantum devices do. Unitaries are what the theory uses to describe some devices. The inference from "this operator is unitary" to "a device does this" needs a device and a mapping. The operator U is unitary for inconsistent wirings. If unitarity were implementation, those wirings would be implemented. They fail the fixed-point test that defines the class. The objection erases the biconditional by treating one side's mathematical vocabulary as if it were a laboratory report.

The third objection says that counterfactual support is all anyone has ever meant by implementation in computer science. Sometimes the word is used that way. Chalmers' condition is stricter, and this paper adopts the stricter use for a stated reason: the looser use counts a correct printout as an implementation of whatever the printout describes. That use would count a printout of a hurricane's wind speeds as a hurricane, which is

the slide his example is meant to stop, whether or not one follows him further to minds. For the wiring, the looser use would count the search's printout as the wiring. Section 3 says why that counting identifies the wrong transitions. If a reader prefers the looser word, the distinction in this paper can be restated without it: the device that prints the count is not a system whose states are the wiring's histories and whose causal transitions are the local maps. The restatement does not need the word "implementation." It needs the two formal structures, which are different.

The fourth objection says that the paper covertly assumes a theory of consciousness by citing Chalmers. The citation is confined to the implementation condition and to the simulation remark. His sufficiency thesis is quoted only to be set aside. No result about consciousness, understanding, agency, or the moral status of a program is stated. A reader who imports one has left the paper. The mathematics would not support the import. A finite biconditional about unitarity and fixed points does not say which properties are organizational invariants.

The fifth objection says that this is the intervention paper or the robustness paper under a new title. The shared mathematics is the same, and the papers are alternative adaptations. The intervention paper asks what a test of a target would have to identify. The robustness paper asks which invariance the theorems are. This paper asks whether a simulator or a unitary description stands in Chalmers' implementation relation to the wiring. A yes to one question is not a yes to the others. In particular, a system can implement the search, which is a yes to a precise implementation question, and still be a no to the question whether the wiring has been realized as a causal structure of histories. That split is the content. It is not a new theorem.

8 Conclusion

Chalmers' condition, as read here, requires a mapping under which formal transitions are reliable causal transitions. The search for fixed points has formal transitions of that shape. A device may implement them. What it then implements is the search. The process-function condition is a fixed-point condition, not that sequence of steps. The canonical permutation is a formal relabeling defined for wirings the fixed-point test fails. It can be the formal half of an implementation claim about a device that permutes states as the relabeling does, and that claim is still not a claim that the device realizes unique histories. A simulator's correct counterfactuals support the model. They do not put the simulator into the implementation relation to the system the model would describe, if there were one.

No thesis about minds is concluded. The condition was used because it says what would have to be true of a physical system, and the public mathematics does not say that about any system. It says when a formal loop is unitary and when a table has one history. Those are the facts. Implementation, simulation, and counterfactual support are three relations one can bear to those facts. Only the first is Chalmers' relation, and only the search is a clear candidate for it, and only as a search.

Use of a language model

Grok Bot, a Cursor cloud agent, drafted this manuscript on 8 November 2026, beyond copy editing. The model version is not recorded. The agent is not an author. Verification is pending. No generative images were produced. The journal page retrieved for this draft states a policy for generative images and requires

statements and declarations; a separate Minds and Machines sentence on generative text, if it exists beyond the publisher's general practice, was not extracted as its own paragraph. The disclosure is made because the drafting was generative.

References

Baumeler, Ä., and S. Wolf. 2016. "The Space of Logically Consistent Classical Processes without Causal Order." *New Journal of Physics* 18:013036. `https://doi.org/10.1088/1367-2630/18/1/013036`

Chalmers, D. J. "A Computational Foundation for the Study of Cognition." `https://consc.net/papers/computation.html`. The note on that page says the text was written in 1993, that section 2 appeared in "On Implementing a Computation," *Minds and Machines* (1994), and that the full text was then forthcoming in the *Journal of Cognitive Science*. The journal printings were not collated. Quotations are from the online text.